

Introduction to Algorithm

기초 알고리즘

그래프 알고리즘 (2)

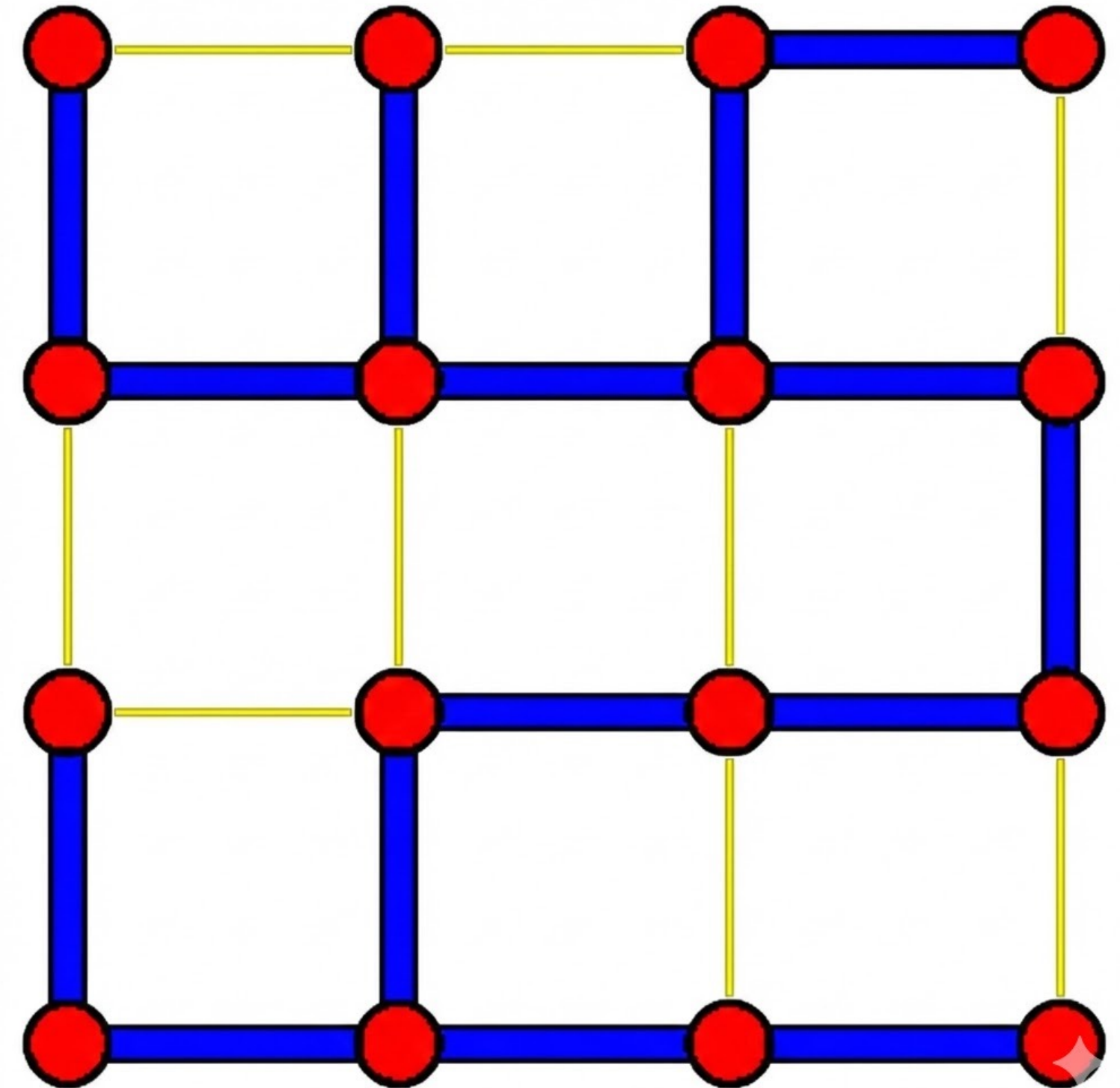
MST



Spanning Tree란?

- 그래프 내의 모든 정점을 포함하는 트리
 - 신장 트리라고도 불림
- N개의 정점을 N-1개의 간선으로 연결
 - 트리 형태이므로 사이클(Cycle) 존재 X

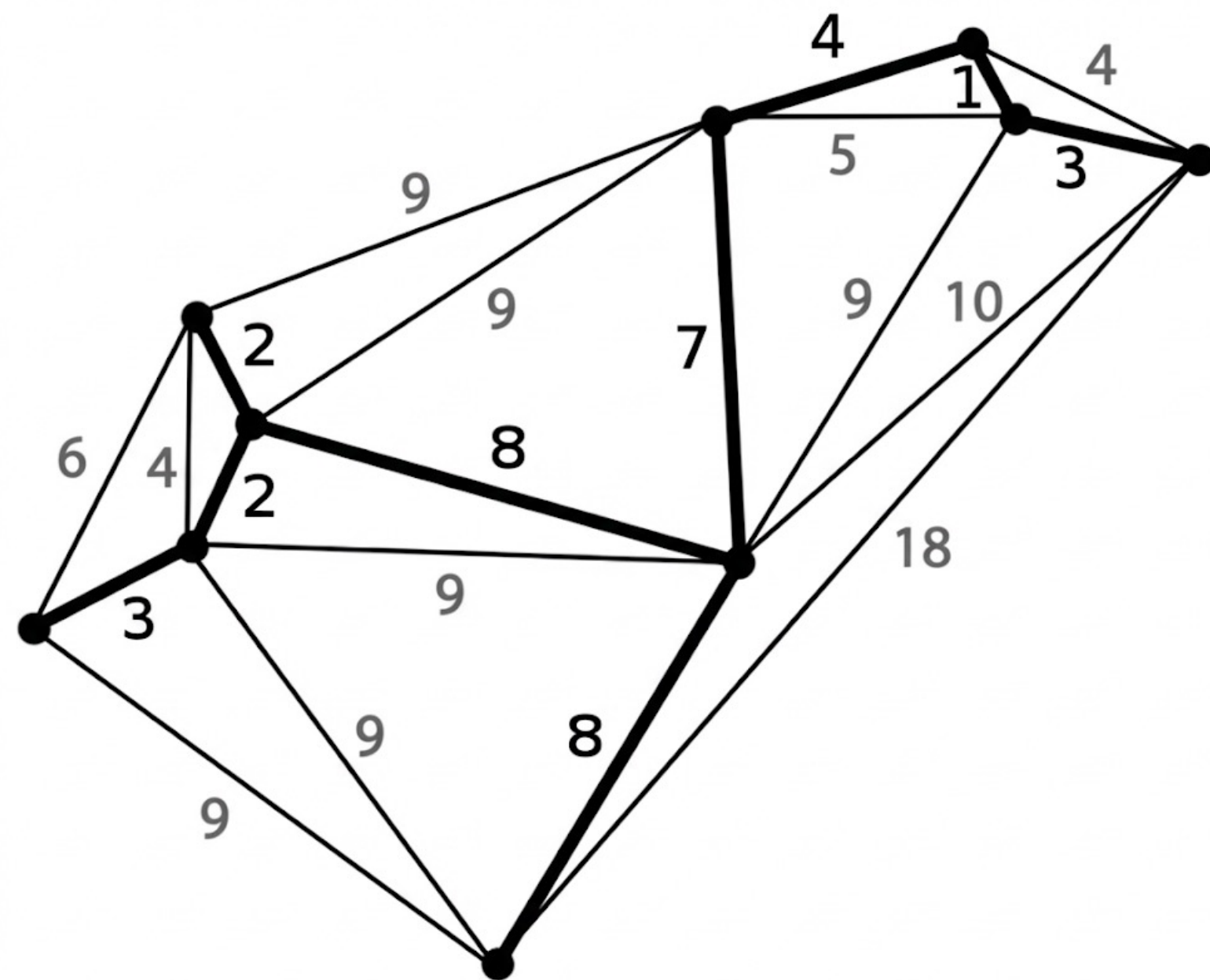
주어진 그래프의 모든 정점을 포함하면서
사이클을 형성하지 않는 최소 연결 부분 그래프





Minimum Spanning Tree란?

- Spanning Tree 중에서 사용된 간선들의 가중치 합이 최소인 트리 (=MST)
- Spanning Tree의 특징을 그대로 가짐
 - N개의 정점을 N-1개의 간선으로 연결
 - 사이클 존재 X
- 주로 통신망, 도로망, 유통망에서 길이, 구축 비용, 전송 시간 등을 최소화하여 구축하려는 경우 사용

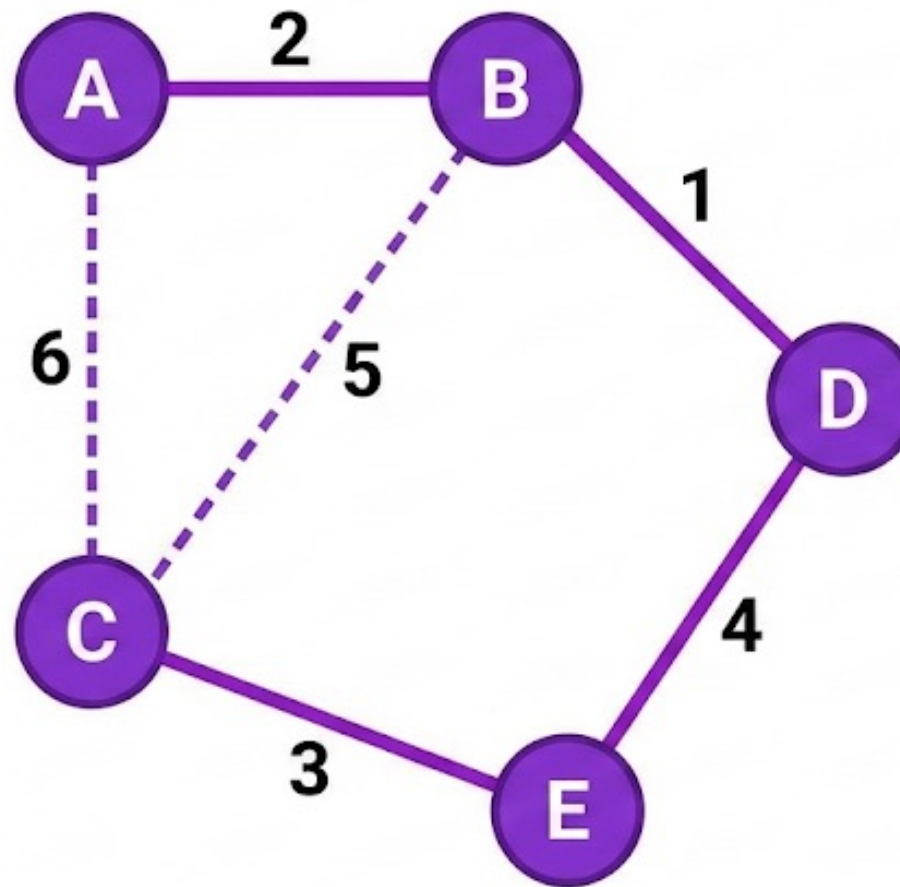




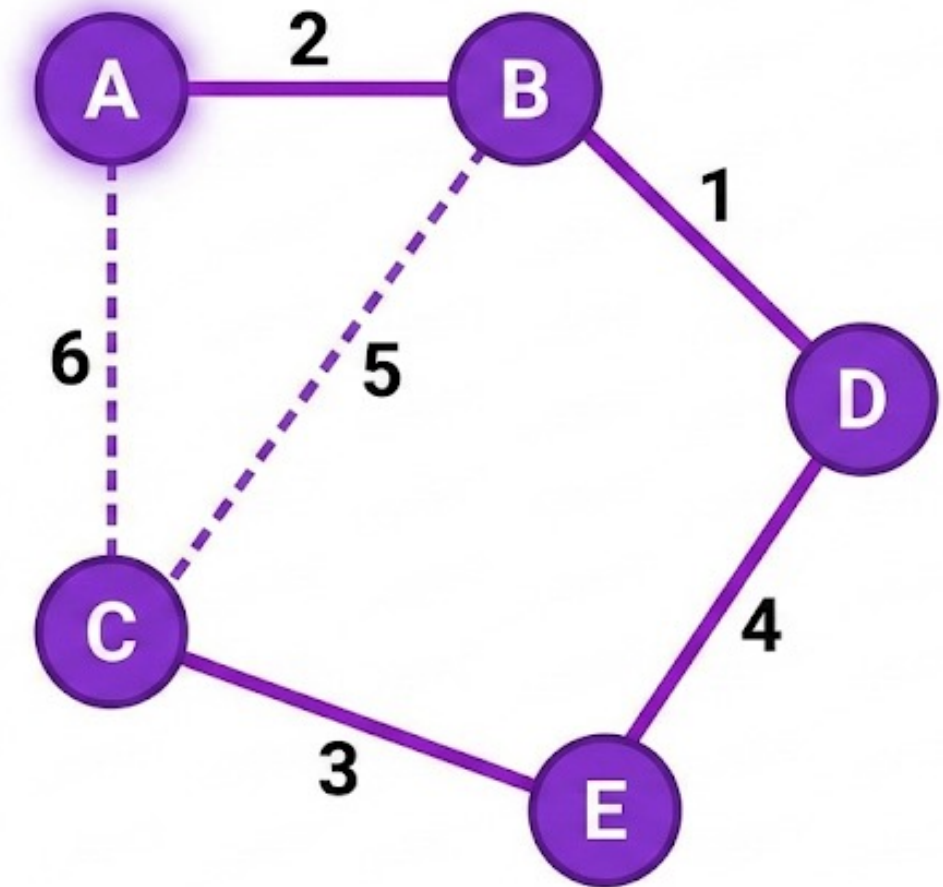
MST 구현 방법 2가지

- 크루스칼 (Kruskal) 알고리즘
 - 간선 중심 알고리즘
- 프림 (Prim) 알고리즘
 - 정점 중심 알고리즘

Kruskal's Algorithm



Prim's Algorithm

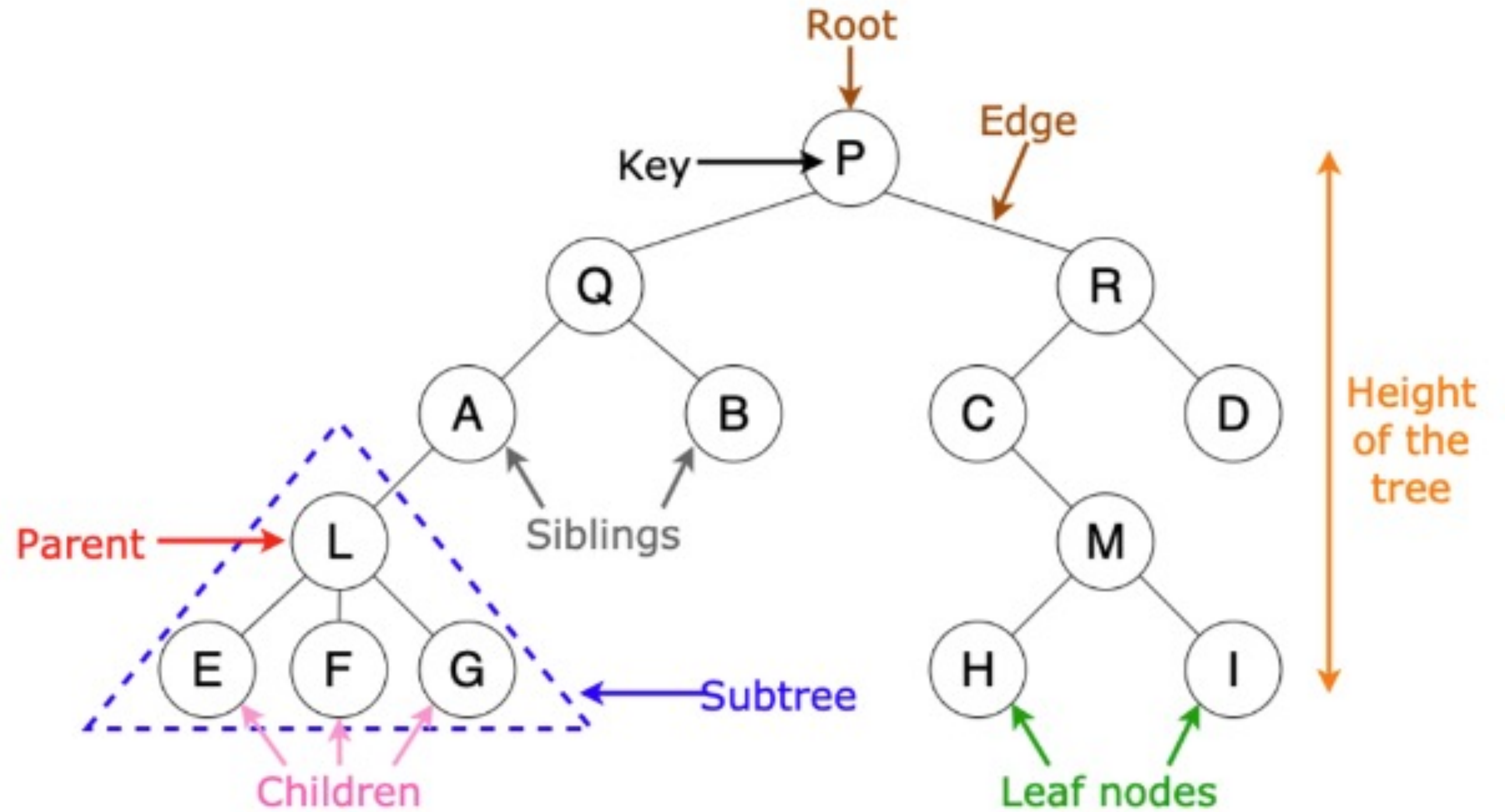


서로소 집합과 유니온 파인드



트리와 관련된 용어들

- 서브 트리
- 깊이
- 루트 노드
- 부모 노드
- 자식 노드
- 형제 노드
- 리프 노드



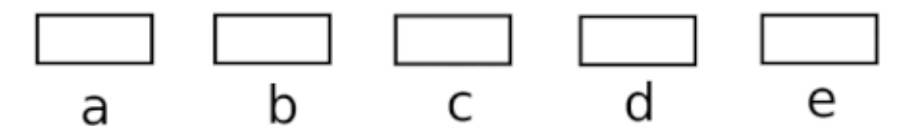
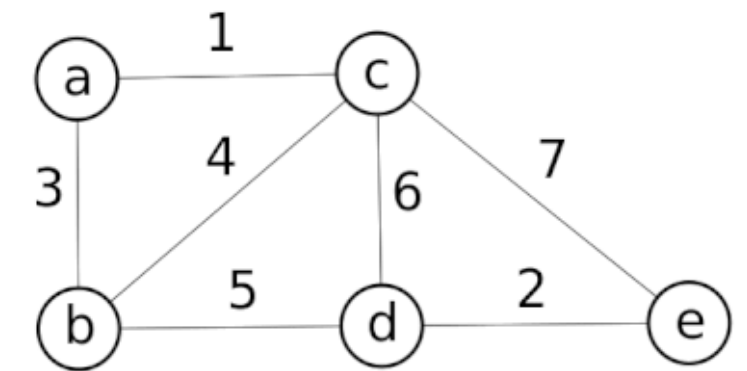
- 서로소 집합 → 서로 공통된 원소를 가지고 있지 않은 두 개 이상의 집합
 - 분리 집합(Disjoint Set)이라고도 부름

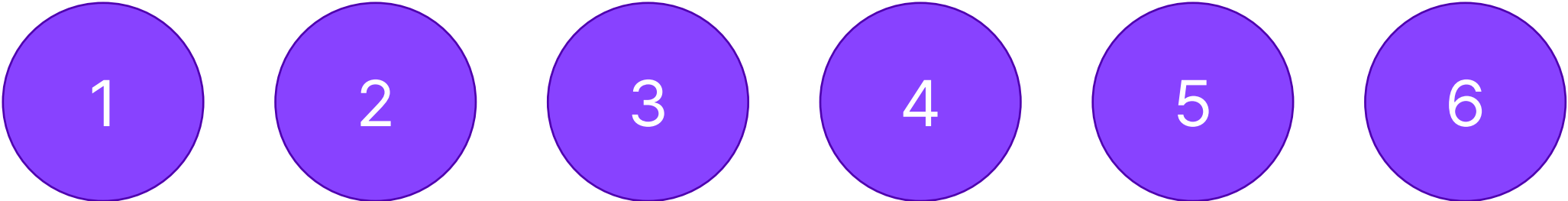
• 사용 용도

- 서로 다른 원소들이 같은 집합에 속해있는지, 아닌지 판별할 때 사용
- 사이클이 존재하는지 판별할 때 사용

• Union-Find 자료구조로 서로소 집합을 표현

- 유니온 파인드가 다른 고급 알고리즘의 베이스가 됨(Kruskal Algorithm)





초기 상태



유니온 파인드의 예시



init()

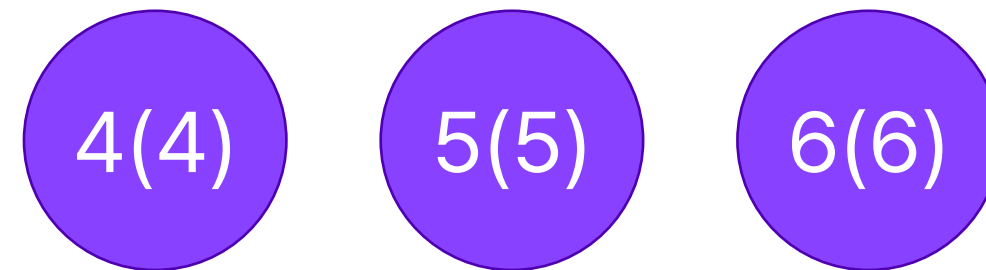
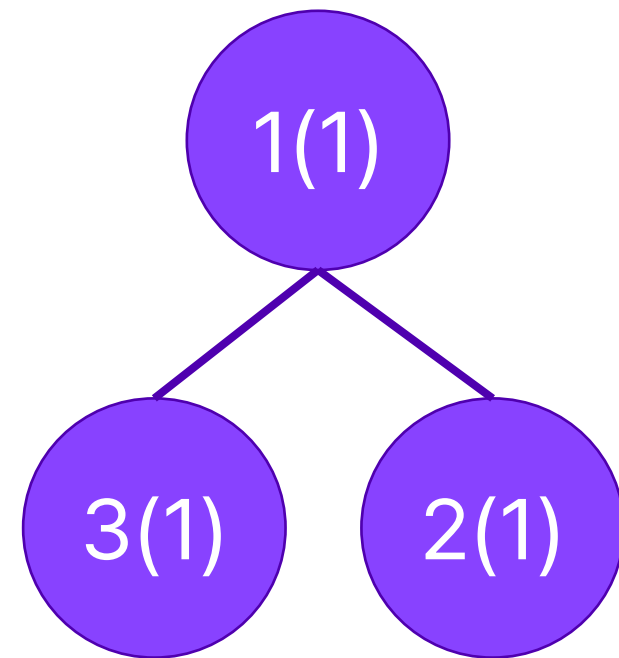


유니온 파인드의 예시





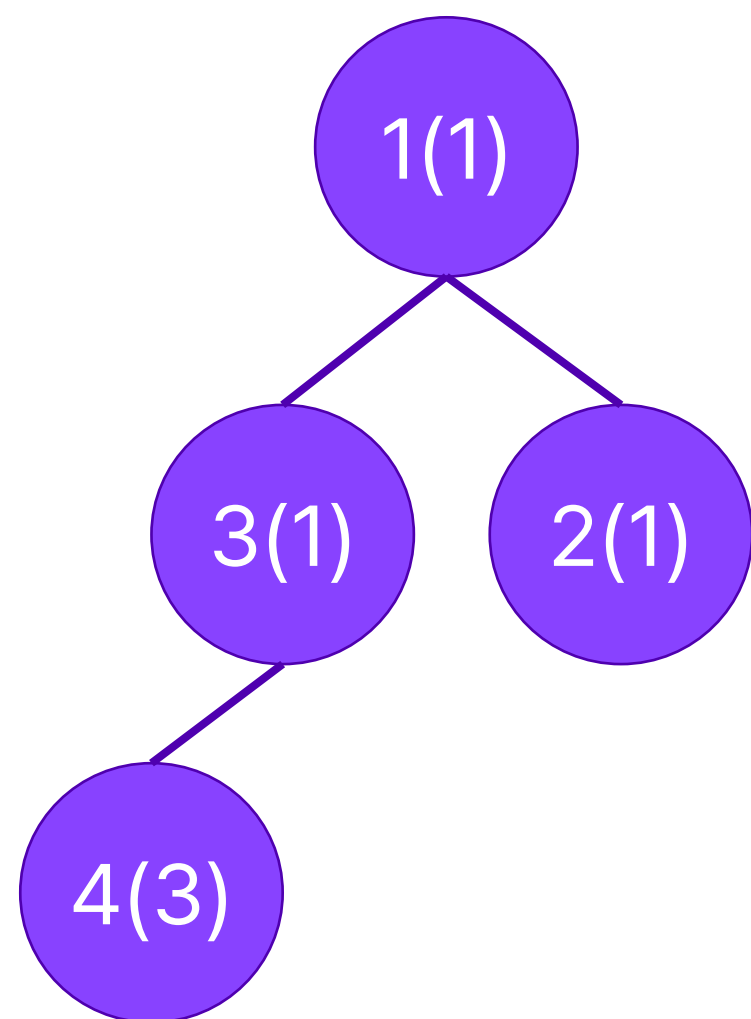
유니온 파인드의 예시



merge_parent(1, 2)



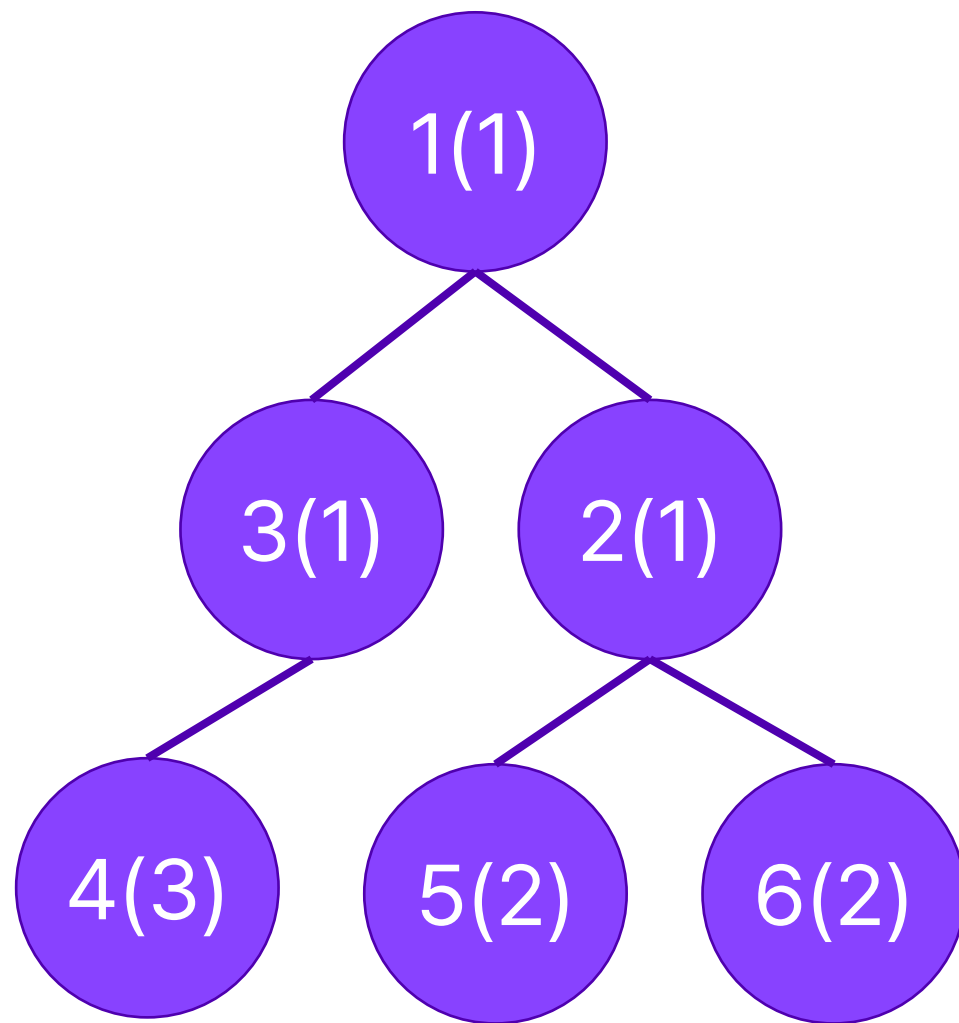
유니온 파인드의 예시



merge_parent(3, 4)



유니온 파인드의 예시



5와 6도 비슷한 방식으로 merge 한다면,
이제 오직 루트 노드만이 자기 자신을 가리키게 됨
→ 이러한 특성을 이용해 루트 노드를 찾을 수 있음



유니온 파인드의 구현

- 유니온 파인드(Union-Find) 자료구조

- init, find, merge(union) 함수들의 형태로 보통 이루어짐
- 함수명 고정 X
- 해당 강의에서는 init, find_parent, merge_parent로 표현

- init

- 초기화 함수
- Parent 배열에 대해 자신의 인덱스 값을 가지도록 초기화
 - 초기에 자신의 부모 노드는 자신이라는 의미

```
def init(n):  
    return list(range(n + 1))  
    # parent[i] = i
```




유니온 파인드의 구현

- find

- 자신의 부모 노드를 찾는 함수
- 재귀 함수로 구현됨
- 자기 자신을 가리키는 인덱스(루트 노드)를 찾을 때까지 반복
- 1보다는 2가 더 빠름(경로 압축)

```
# parent는 1~n 인덱스를 쓰는 리스트라고 가정
def find_parent1(parent, x):
    """경로 압축 없음"""
    return x if parent[x] == x else find_parent1(parent, parent[x])

def find_parent2(parent, x):
    """경로 압축(path compression) 적용"""
    if parent[x] == x:
        return x
    parent[x] = find_parent2(parent, parent[x]) #parent[x]를 바로 변경
    return parent[x]
```



유니온 파인드의 구현

- find
 - 3은 2를 재귀함수를 사용하지 않고 구현
 - 1보다는 빠르고, 2와 비슷

```
def find_parent3_iter(parent, x):  
    root = x  
  
    # 루트 탐색  
    while parent[root] != root:  
        root = parent[root]  
  
    # 경로 압축  
    while parent[x] != x:  
        nxt = parent[x]  
        parent[x] = root  
        x = nxt  
    return root
```



유니온 파인드의 구현

- merge
 - 두 노드를 하나의 집합으로 합치는 함수
 - x 의 부모 노드는 y
 - find 함수를 같이 사용
 - if 문에서 $x == y$ 이면?
 - 사이클이 발생하는 경우이므로 제외

```
def merge_parent1(parent, x, y):  
    # 경로 압축 find 사용(원하면 find_parent1도 가능)  
    x = find_parent2(parent, x)  
    y = find_parent2(parent, y)  
  
    if x != y:  
        parent[y] = x
```



유니온 파인드의 구현

- merge
 - 집합의 크기를 기준으로 병합
 - 트리 높이를 줄여 성능을 개선
 - 1은 한쪽으로 치우친 트리가 생성되어 높이가 선형적으로 증가할 수 있음

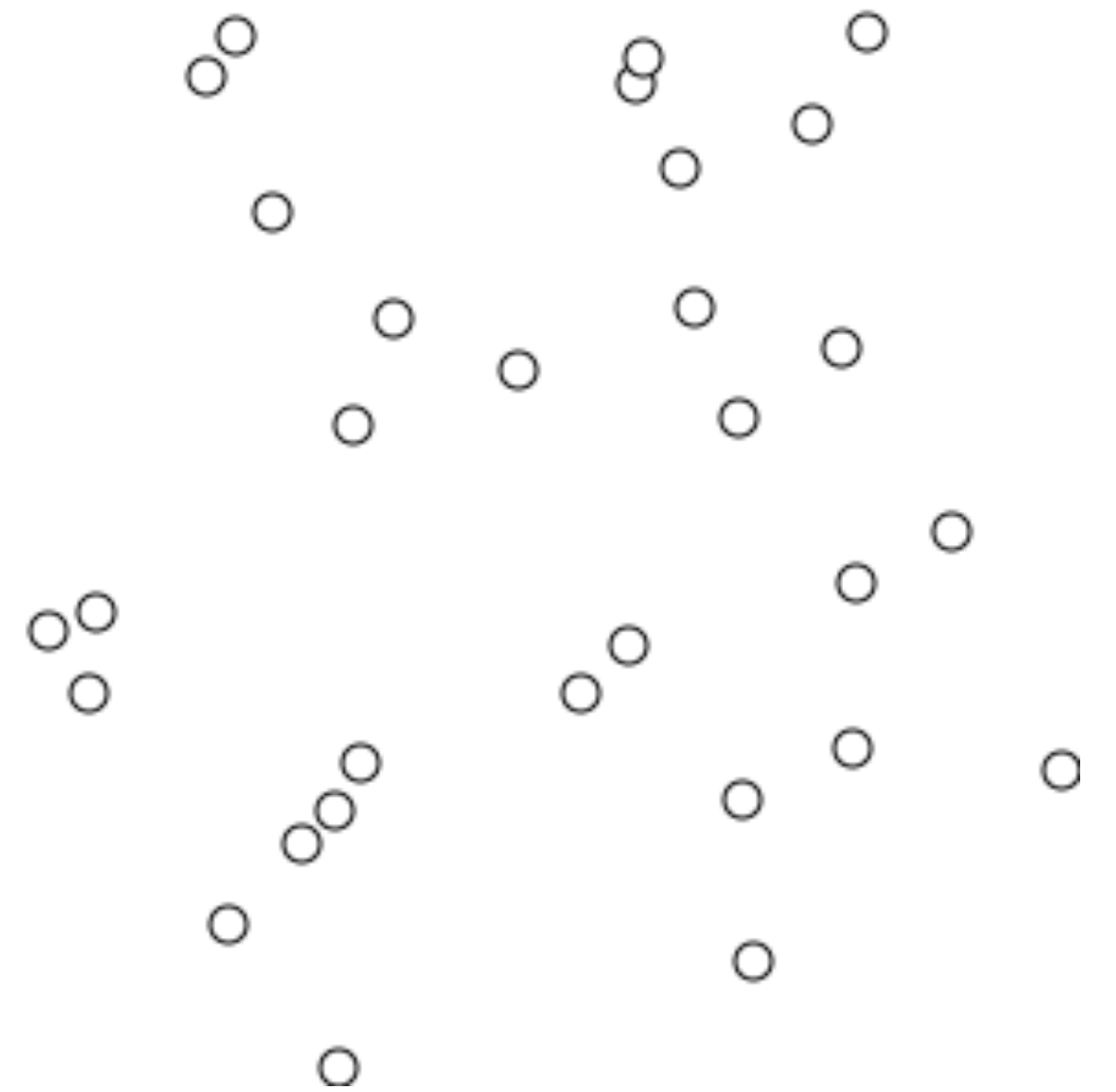
```
def merge_parent2(parent, size, x, y):  
    x = find_parent2(parent, x)  
    y = find_parent2(parent, y)  
    if x == y:  
        return False  
  
    # 작은 트리를 큰 트리 밑으로  
    if size[x] < size[y]:  
        x, y = y, x  
    parent[y] = x  
    size[x] += size[y]  
    return True
```


Kruskal



크루스칼 알고리즘의 특징

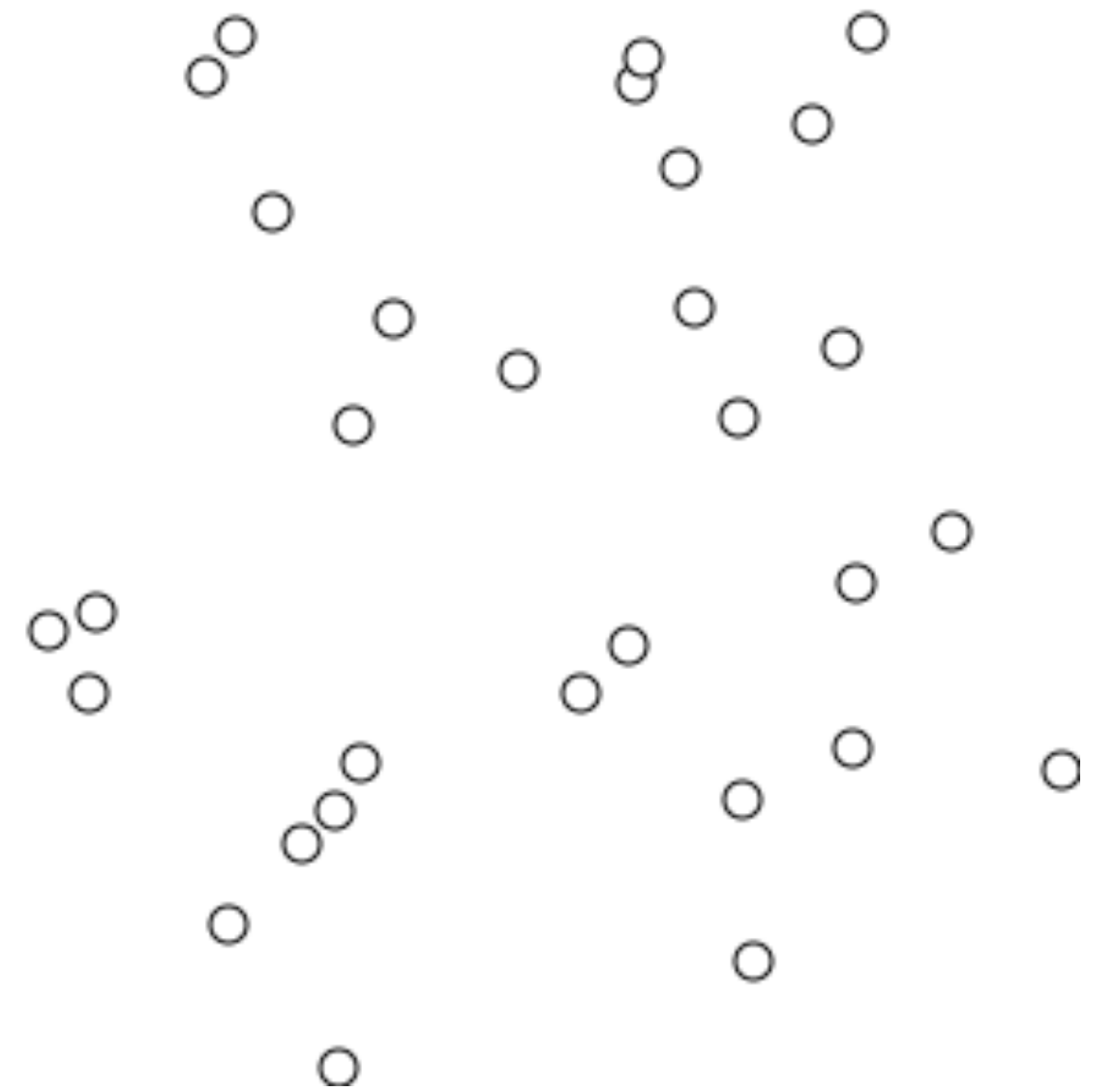
- 그리디 (Greedy) 알고리즘을 이용
 - 가장 비용이 작은 간선부터 선택
- 유니온 파인드 자료구조 활용
 - 사이클 검사
- 시간 복잡도 : $O(E \log E)$





크루스칼 알고리즘의 동작 방식

1. 가중치의 오름차순 기준으로 간선들을 정렬
2. 가중치가 낮은 간선부터 연결
 - 단, 연결 시 사이클이 존재하게 된다면 그 간선은 연결하지 않음
 - 유니온 파인드 기법으로 확인 가능
 - 즉, 이으려는 두 정점 모두가 이미 연결되어 있는 상태일 때는 연결하지 않음
3. 2번을 모든 정점이 연결될 때까지 반복

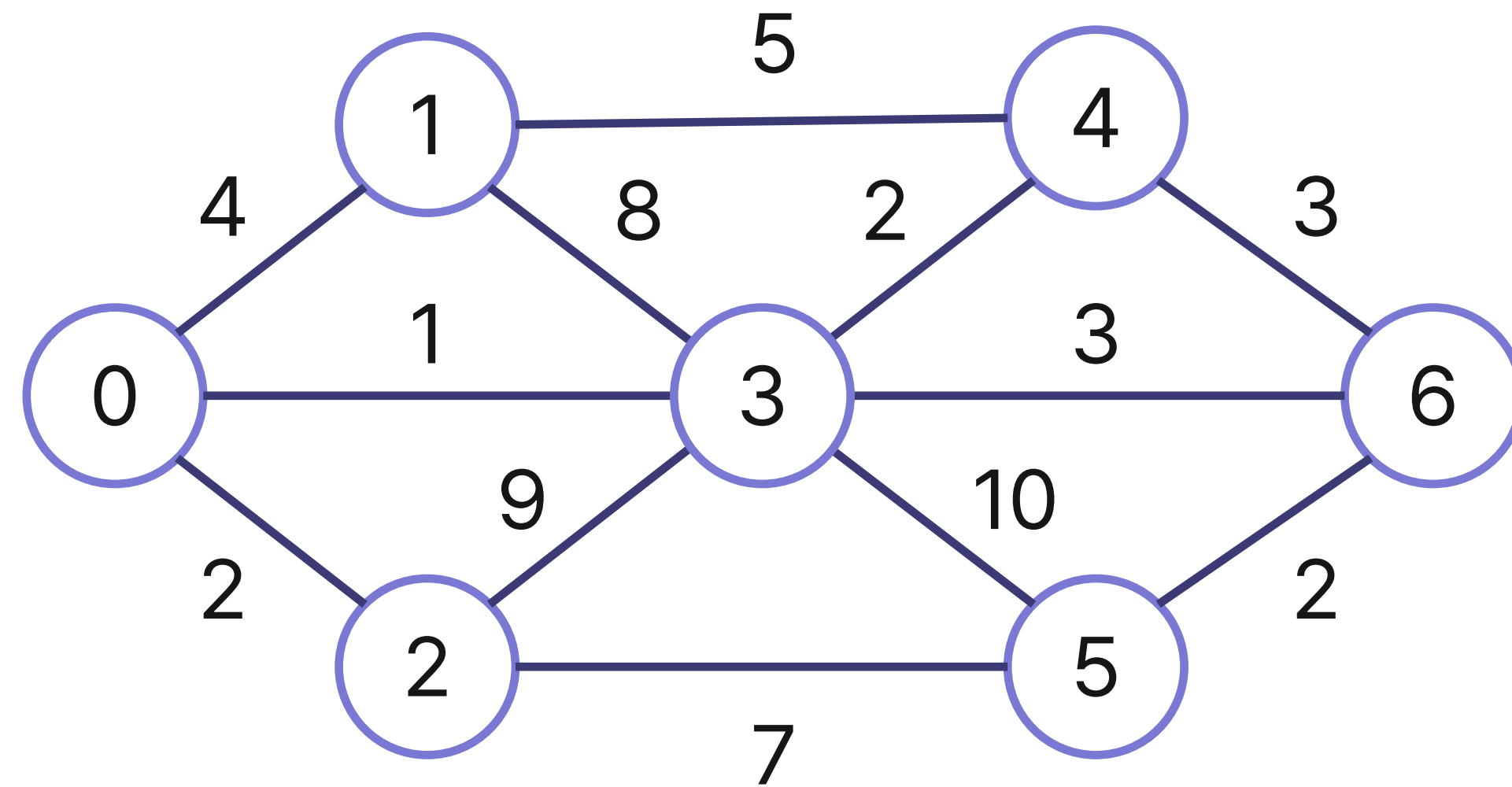




크루스칼 (Kruskal)

짧은 간선부터 골라 그래프에 포함시키는 방법

단, 트리에는 사이클이 없기 때문에, 사이클이 되는 경우는 건너뜁니다.

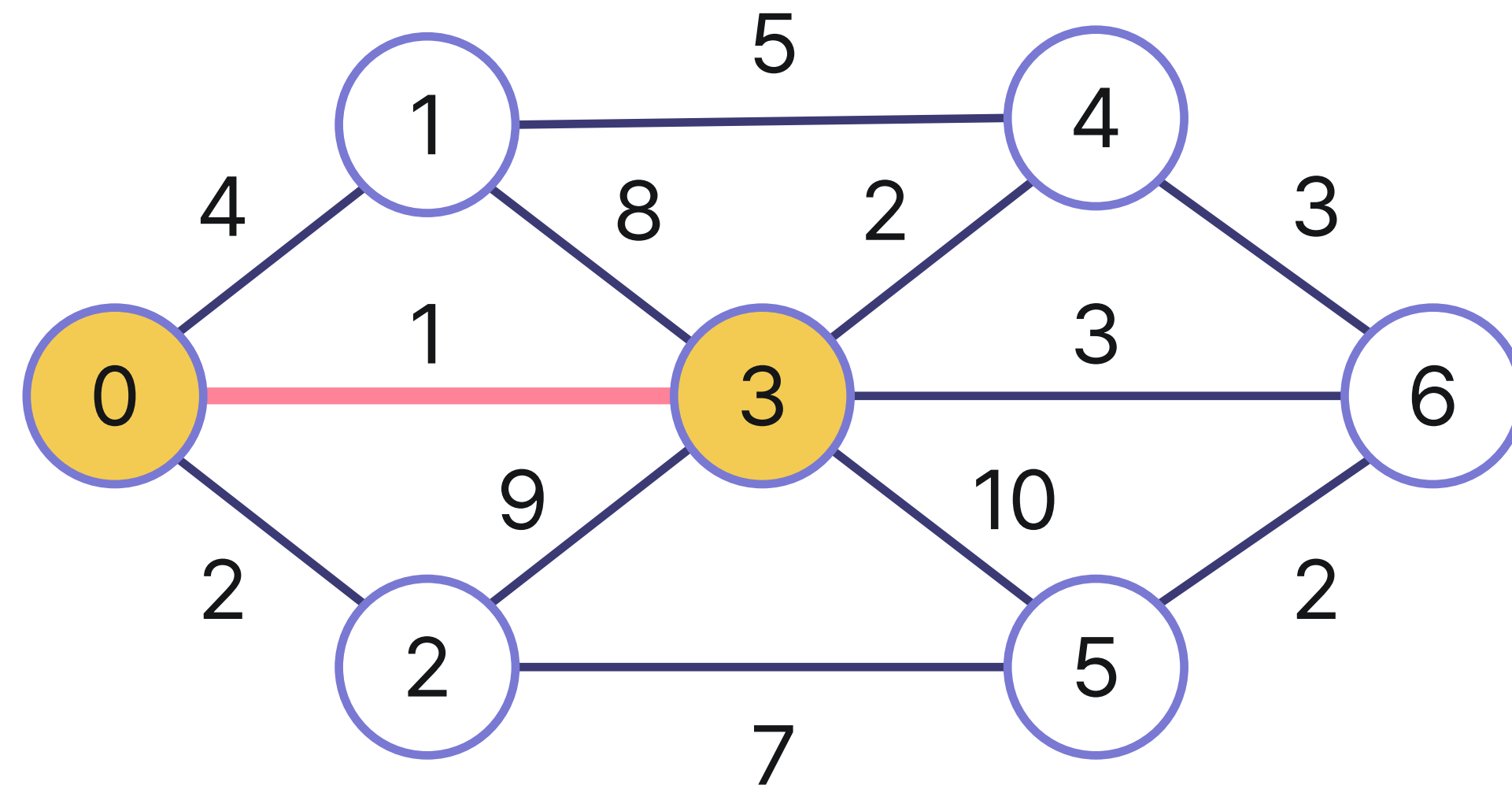




크루스칼 (Kruskal)

짧은 간선부터 골라 그래프에 포함시키는 방법

단, 트리에는 사이클이 없기 때문에, 사이클이 되는 경우는 건너뜁니다.

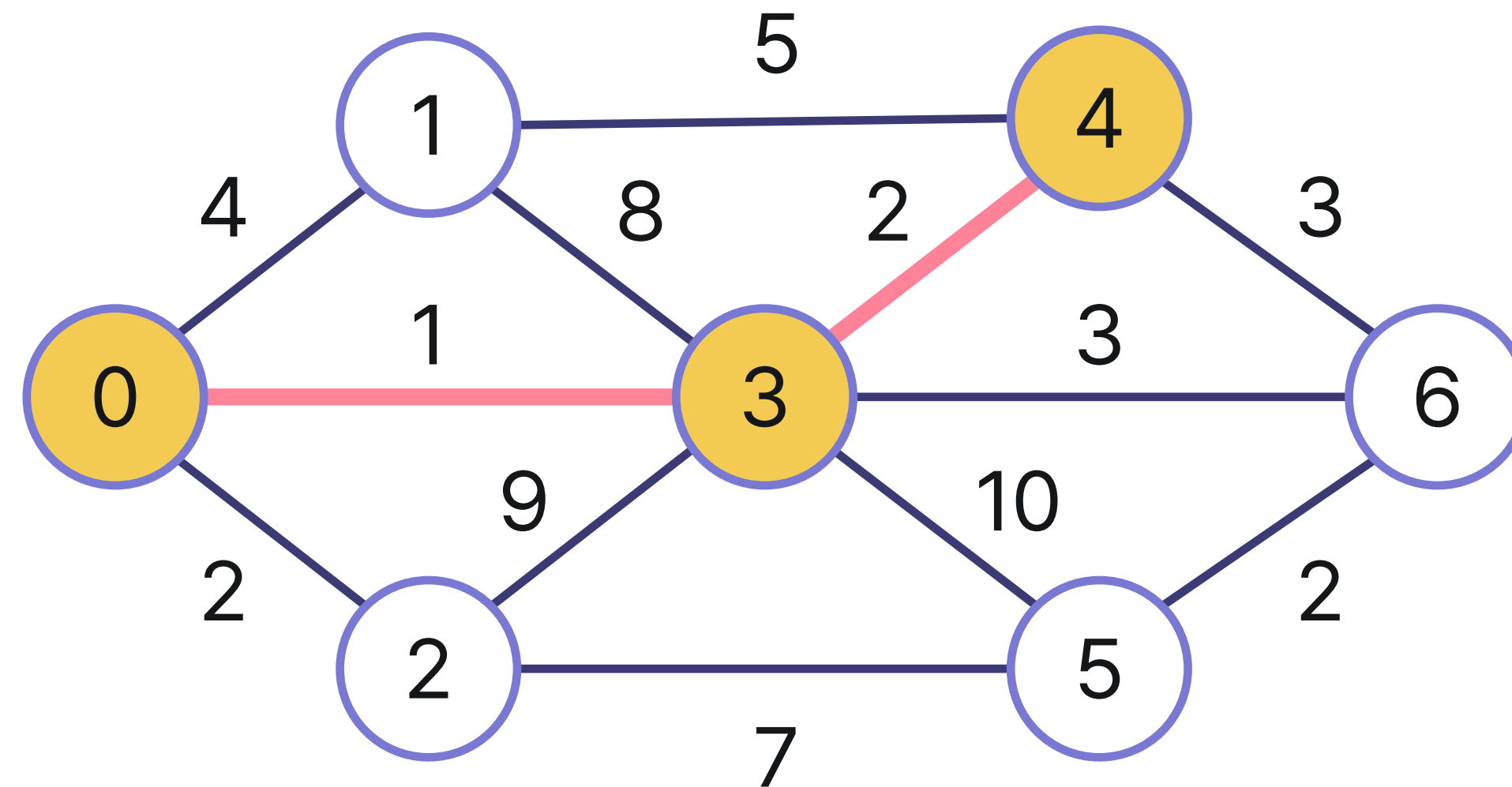




크루스칼 (Kruskal)

짧은 간선부터 골라 그래프에 포함시키는 방법

단, 트리에는 사이클이 없기 때문에, 사이클이 되는 경우는 건너뜁니다.

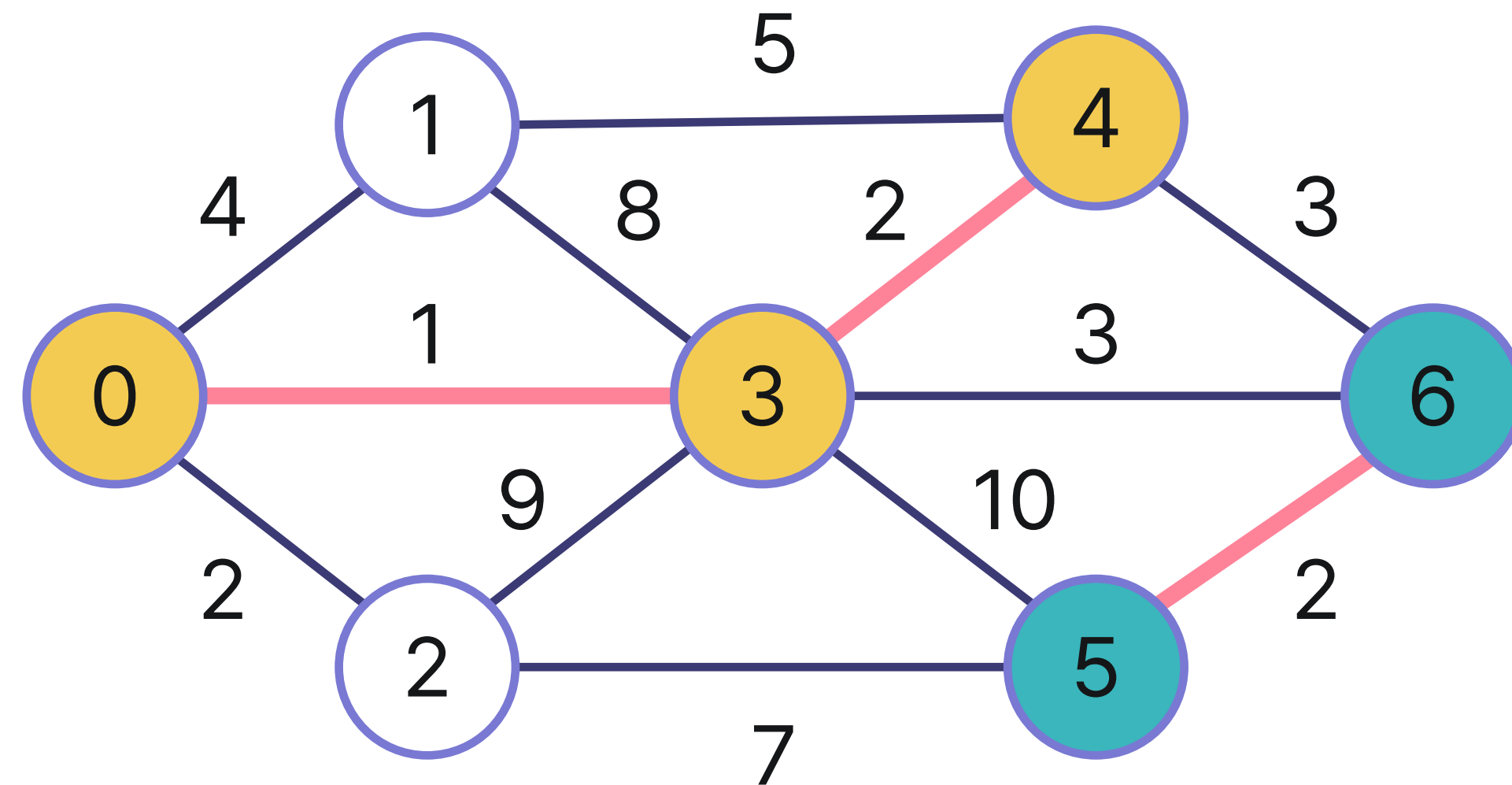




크루스칼 (Kruskal)

짧은 간선부터 골라 그래프에 포함시키는 방법

단, 트리에는 사이클이 없기 때문에, 사이클이 되는 경우는 건너뜁니다.

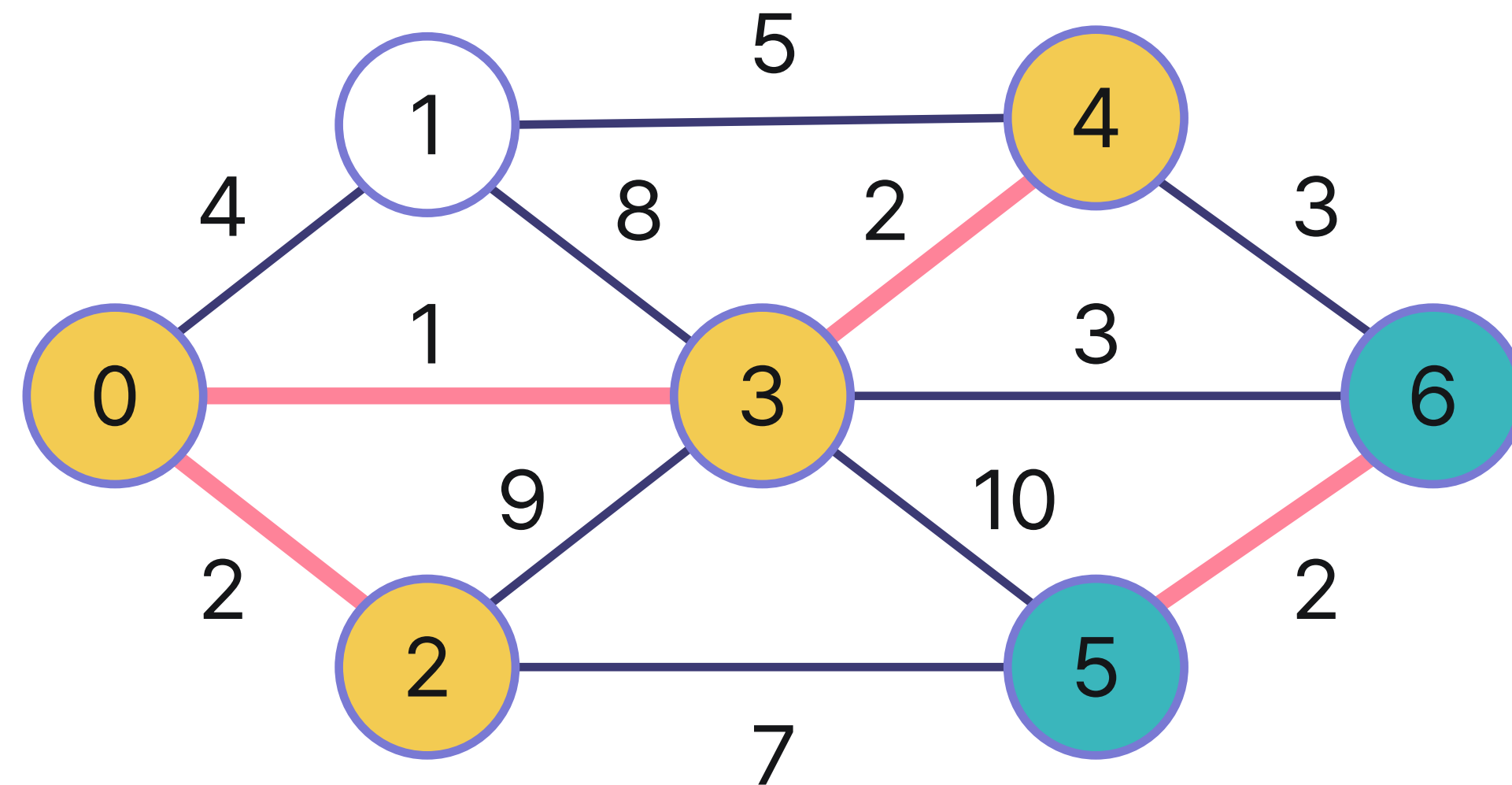




크루스칼 (Kruskal)

짧은 간선부터 골라 그래프에 포함시키는 방법

단, 트리에는 사이클이 없기 때문에, 사이클이 되는 경우는 건너뜁니다.

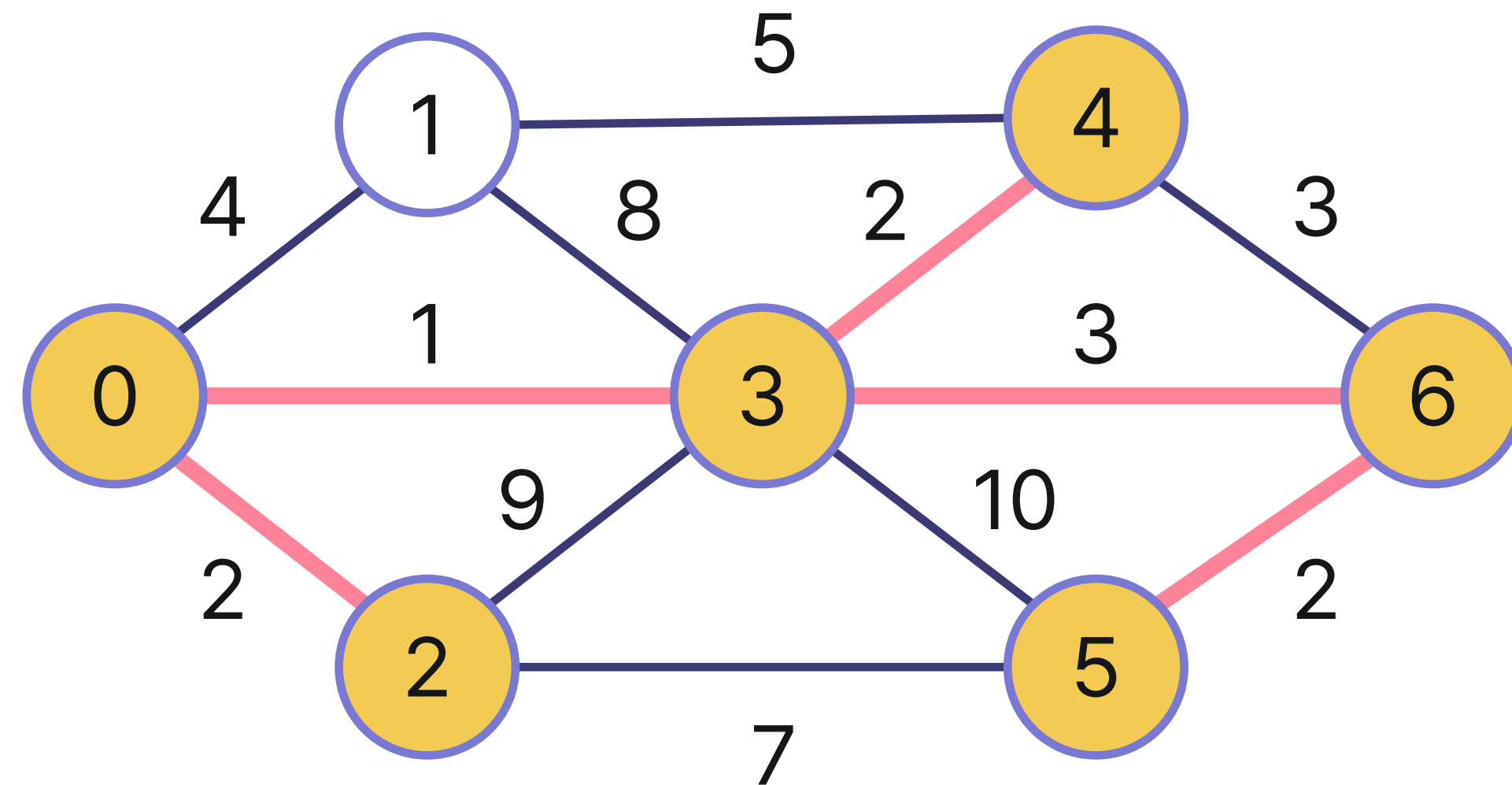




크루스칼 (Kruskal)

짧은 간선부터 골라 그래프에 포함시키는 방법

단, 트리에는 사이클이 없기 때문에, 사이클이 되는 경우는 건너뜁니다.

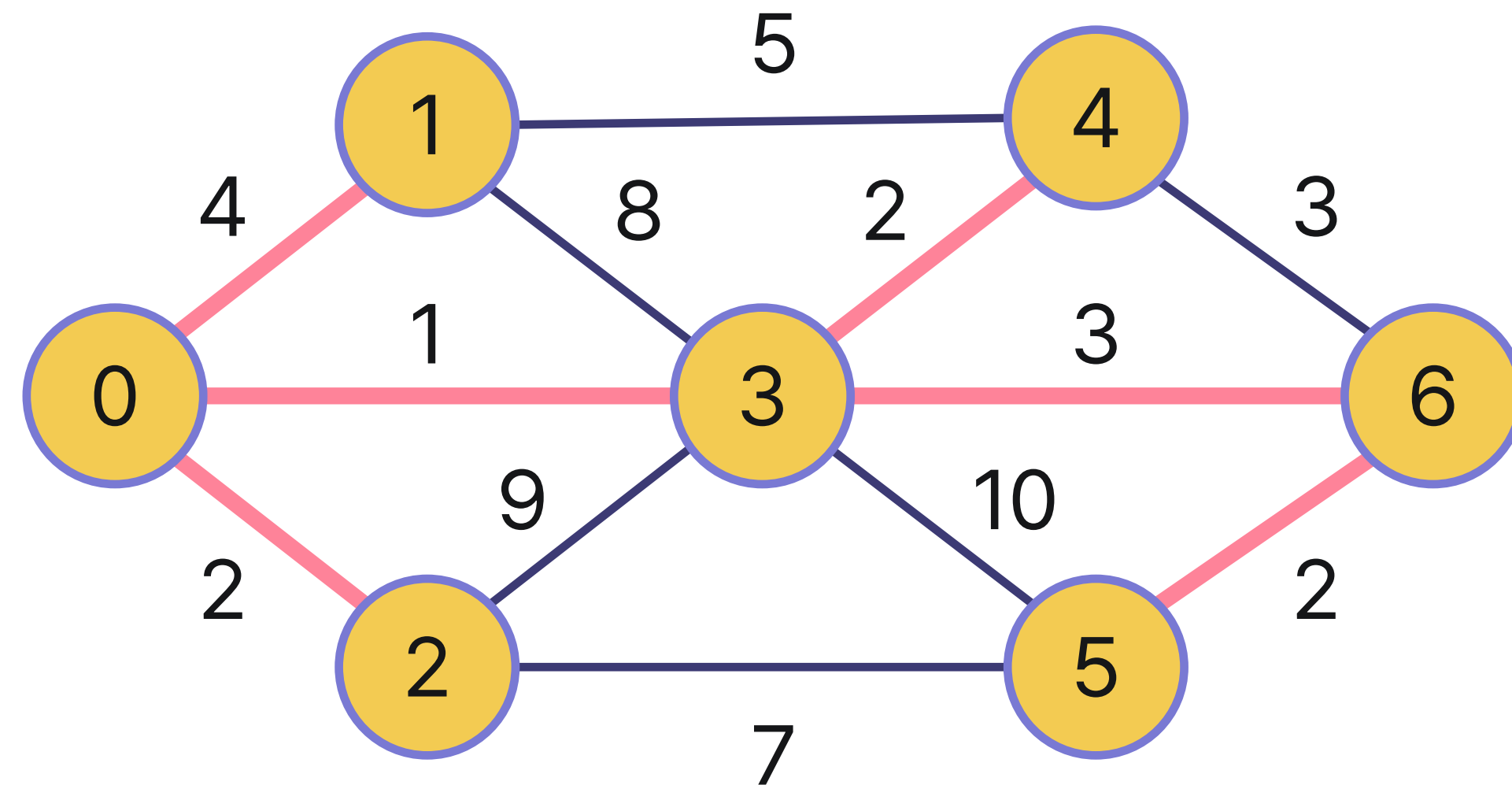




크루스칼 (Kruskal)

짧은 간선부터 골라 그래프에 포함시키는 방법

단, 트리에는 사이클이 없기 때문에, 사이클이 되는 경우는 건너뜁니다.





N개의 정점과 M개의 간선으로 이루어진 무방향 그래프가 주어질 때, 주어진 그래프의 모든 정점들을 가중치의 합이 최소가 되도록 MST를 만드려고 한다.

위 조건대로 모든 정점들을 연결하였을 때의 가중치를 출력하는 프로그램을 작성해보자.



크루스칼 (Kruskal)

1. 가중치의 오름차순 기준으로 간선들을 정렬

```
edges = []  
for i in range(m):  
    a, b, c = map(int, input().split())  
    edges.append((c, (a, b)))  
edges.sort()
```



크루스칼 (Kruskal)

2. 가중치가 낮은 간선부터 연결
3. 2번을 모든 정점이 연결될 때까지 반복

```
cnt = 0
sum_value = 0

for i in range(m):
    from_node, to_node = edges[i][1]
    if find_parent(from_node) != find_parent(to_node):
        union_parent(from_node, to_node)
        cnt += 1
        sum_value += edges[i][0]
    if cnt == n - 1:
        break

print(sum_value)
```

Prim



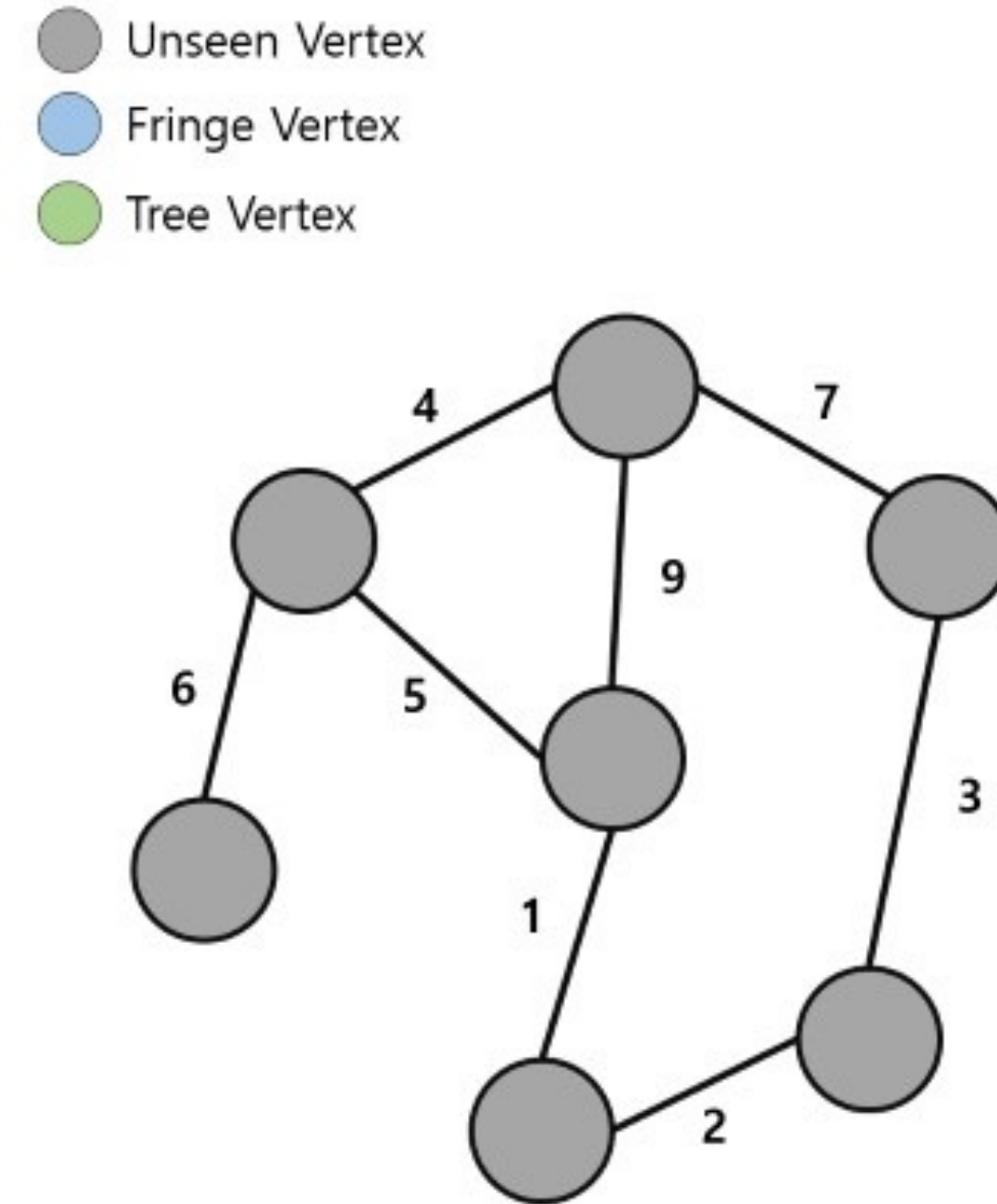
프림 알고리즘의 특징

- 시작 정점에서 출발하여 현재 포함된 정점들과 연결된 간선 중에서 최소 가중치를 가진 간선을 선택하여 트리에 추가
- 정점이 기준이 되는 알고리즘
- 시간 복잡도 : $O(E \log V)$ (우선순위 큐)



프림 알고리즘의 동작 방식

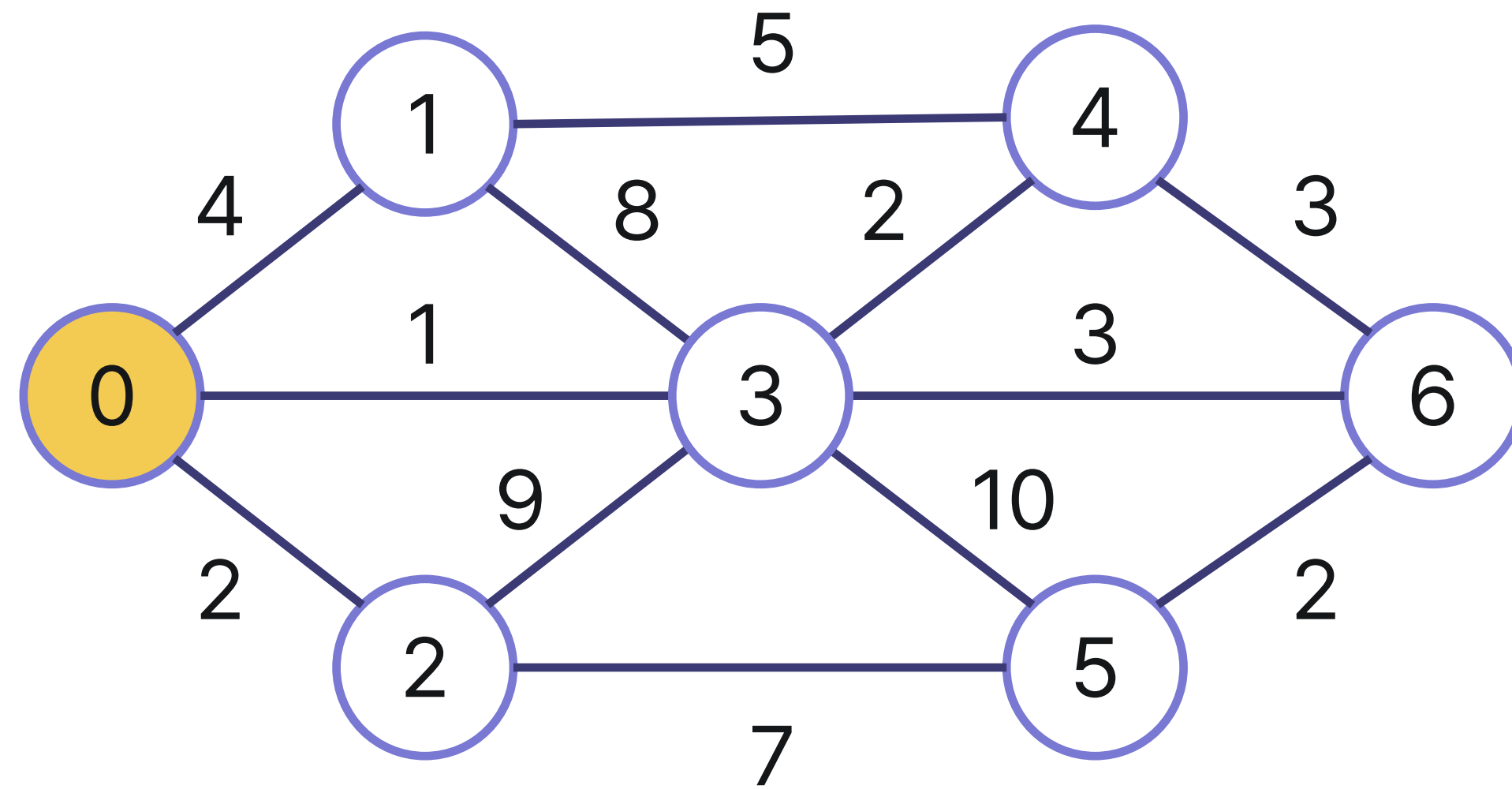
1. 시작 정점만을 MST 집합에 포함
2. 이후, MST 집합에 인접한 정점들 중에서 가중치가 가장 작은 간선으로 연결된 정점을 선택하여 트리를 확장
3. 2번을 모든 정점이 연결될 때까지 반복





프림 (Prim)

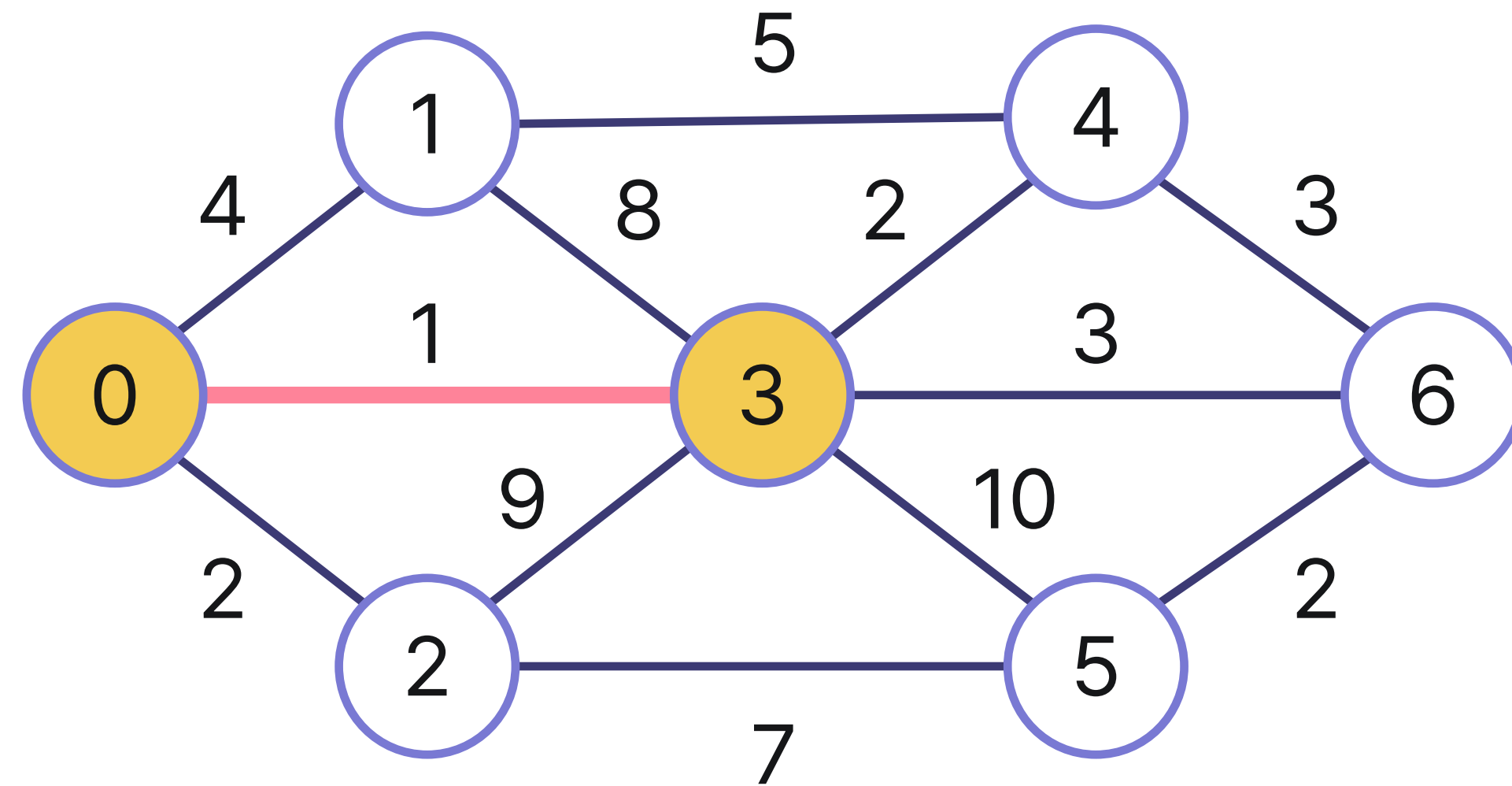
작은 트리에서 연결되어있는 간선들을 하나씩 확장하면서 트리를 만듭니다.





프림 (Prim)

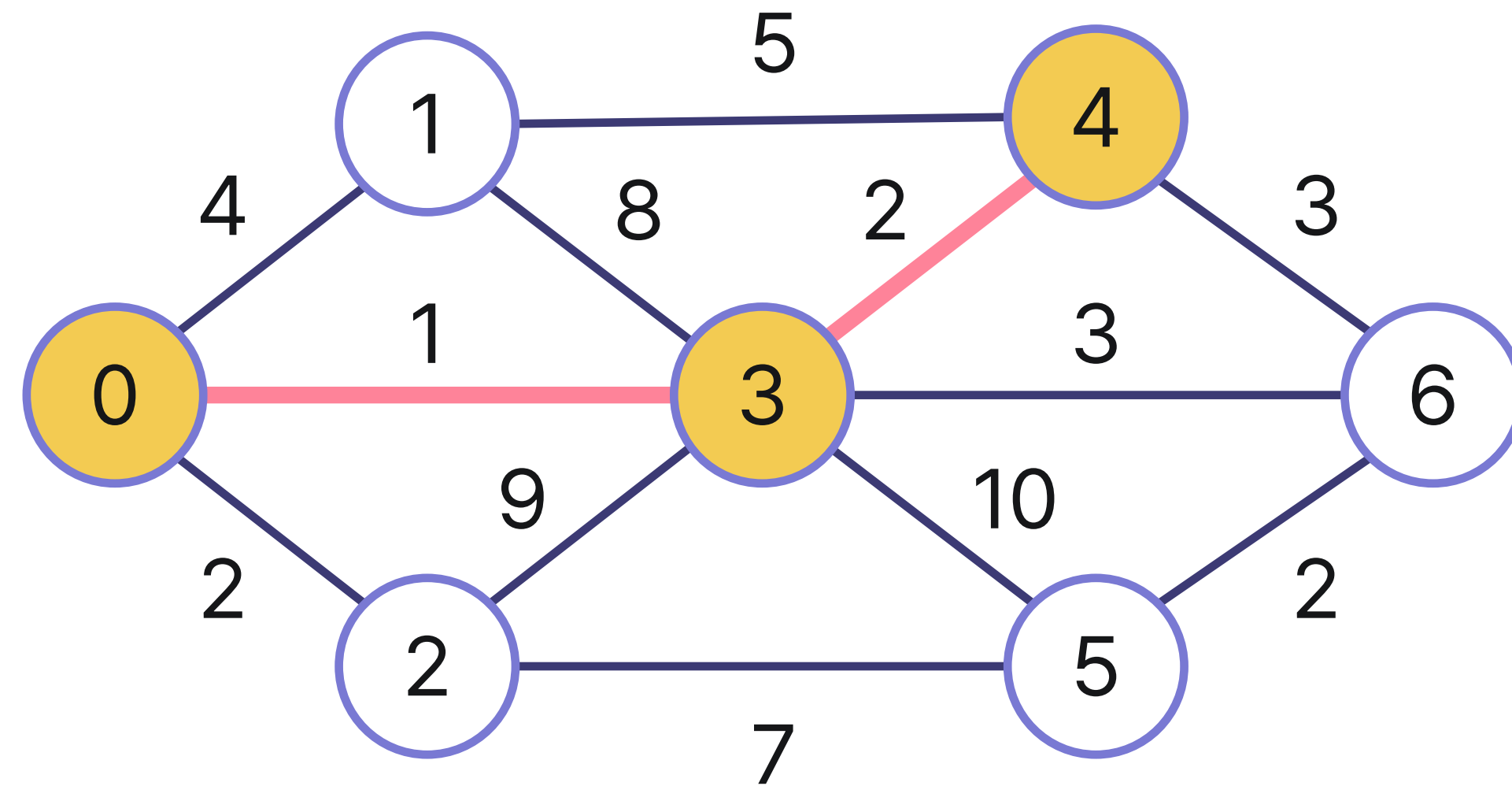
작은 트리에서 연결되어있는 간선들을 하나씩 확장하면서 트리를 만듭니다.





프림 (Prim)

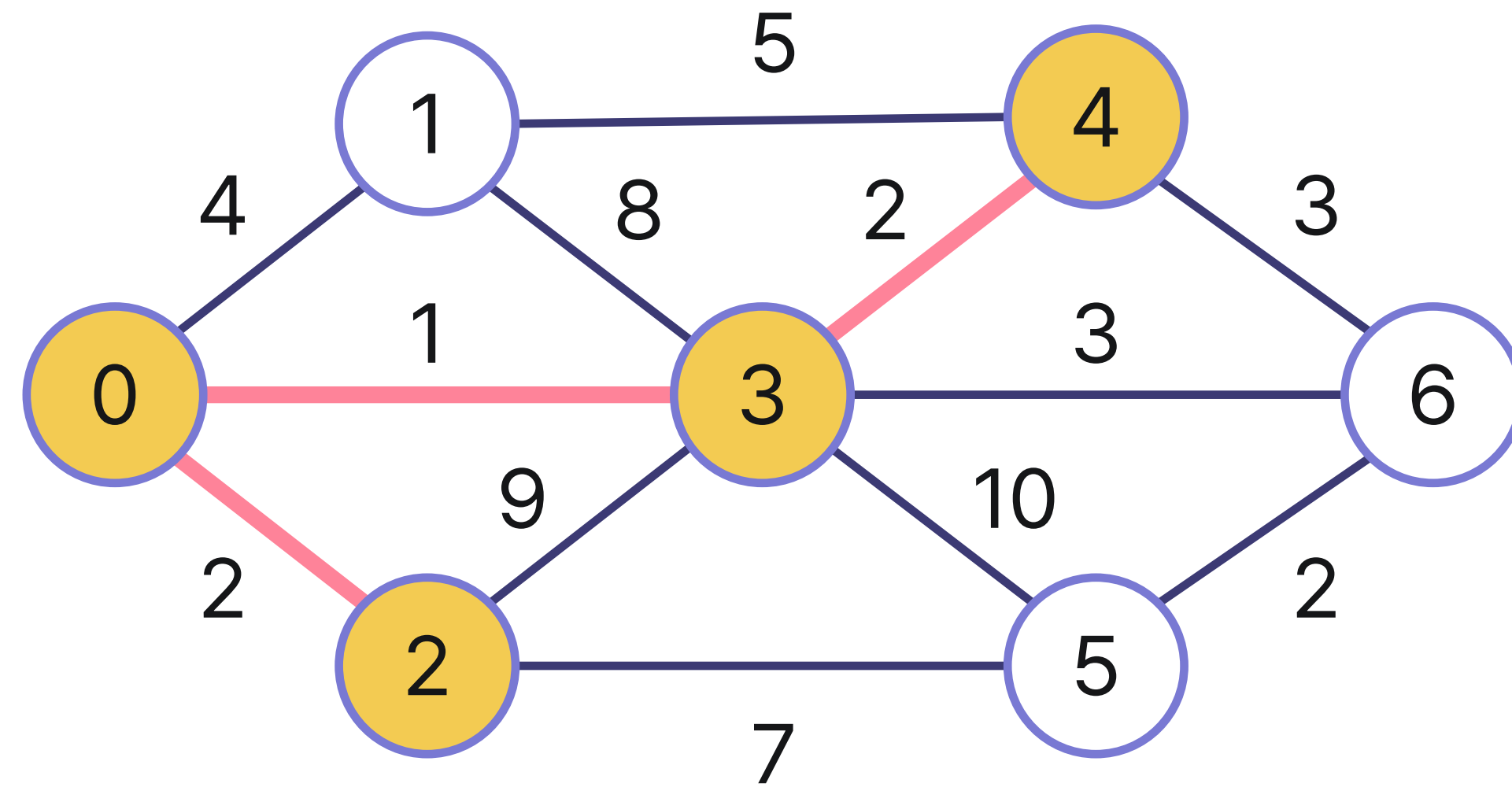
작은 트리에서 연결되어있는 간선들을 하나씩 확장하면서 트리를 만듭니다.





프림 (Prim)

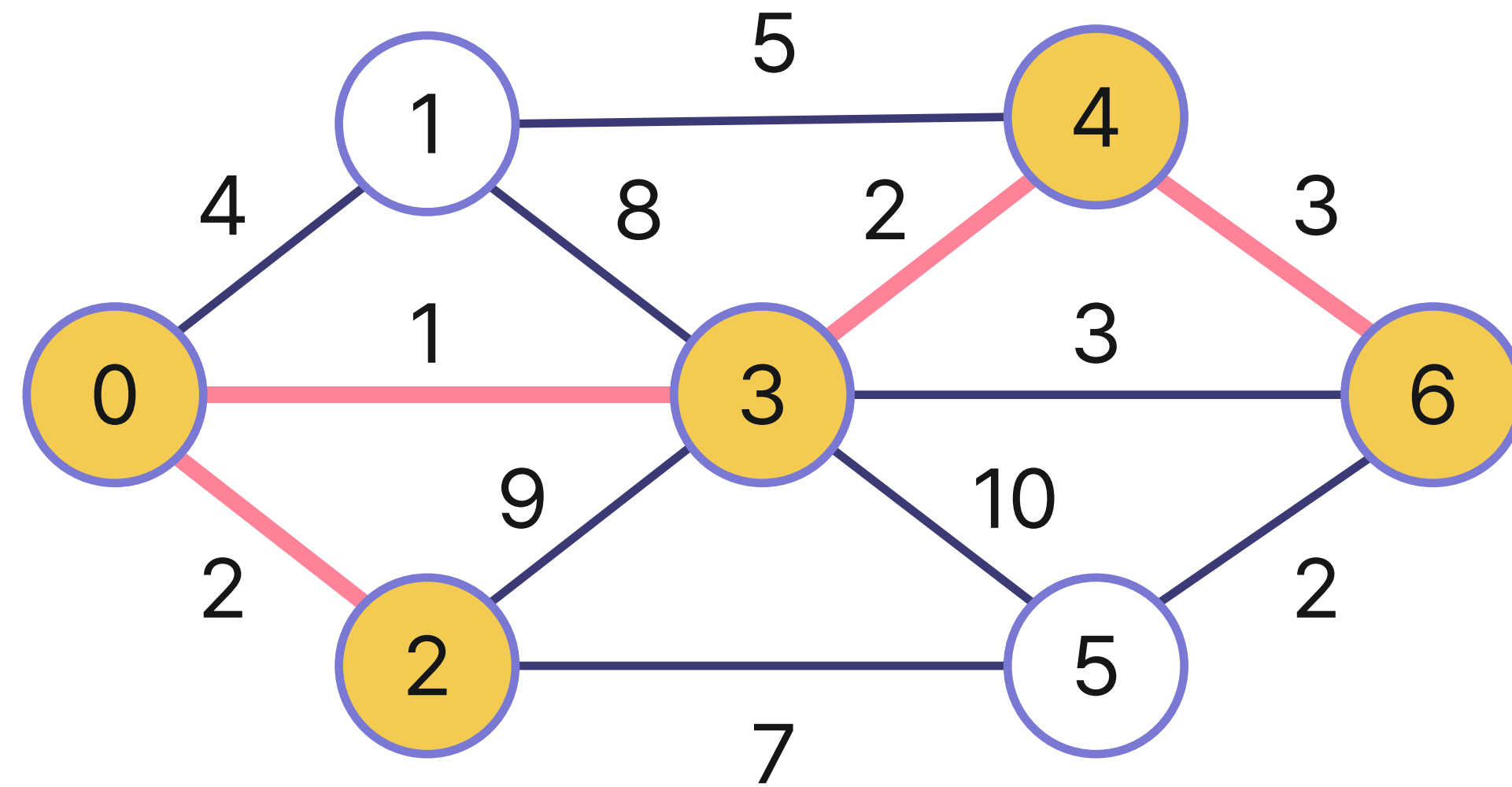
작은 트리에서 연결되어있는 간선들을 하나씩 확장하면서 트리를 만듭니다.





프림 (Prim)

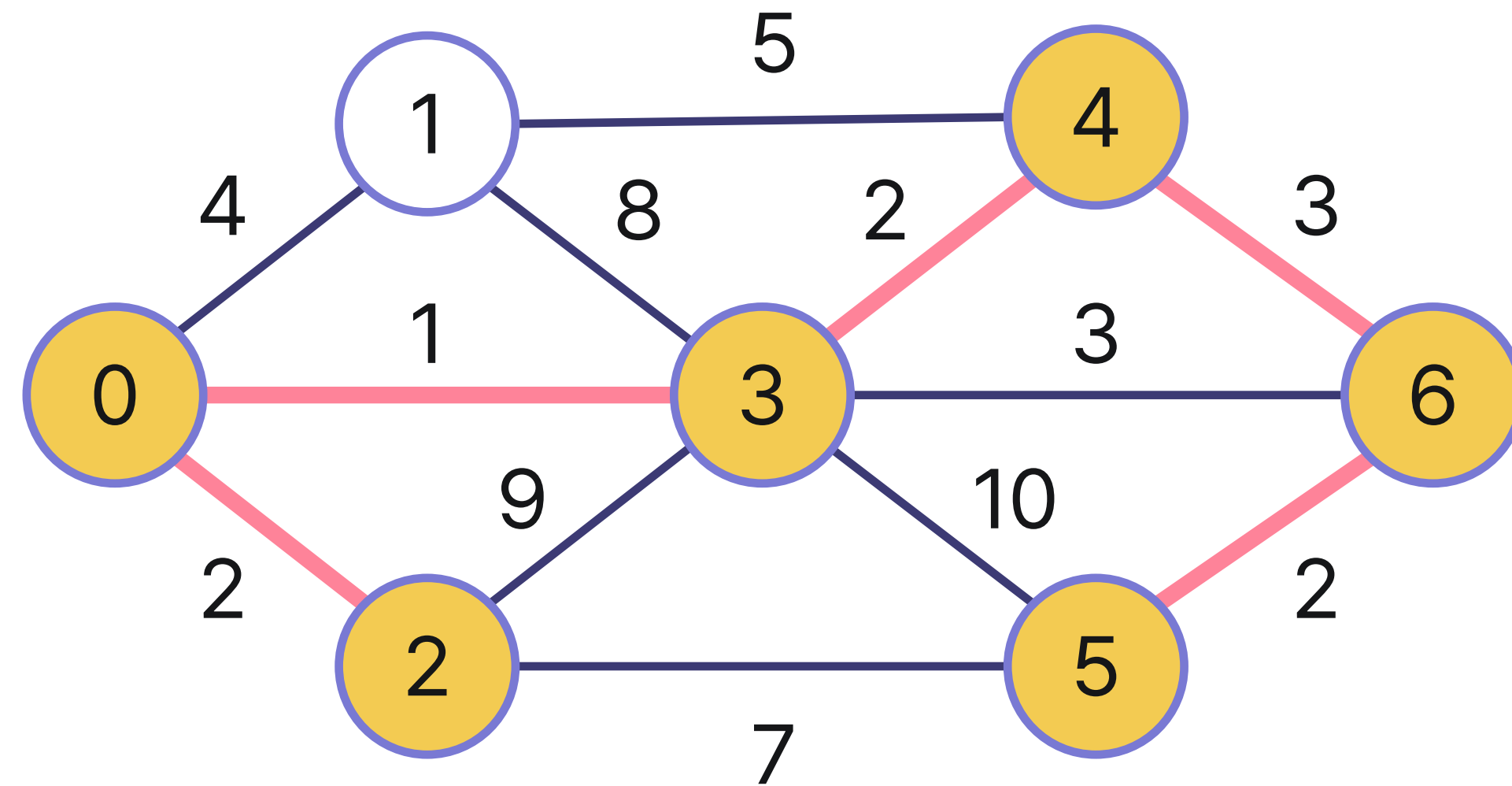
작은 트리에서 연결되어있는 간선들을 하나씩 확장하면서 트리를 만듭니다.





프림 (Prim)

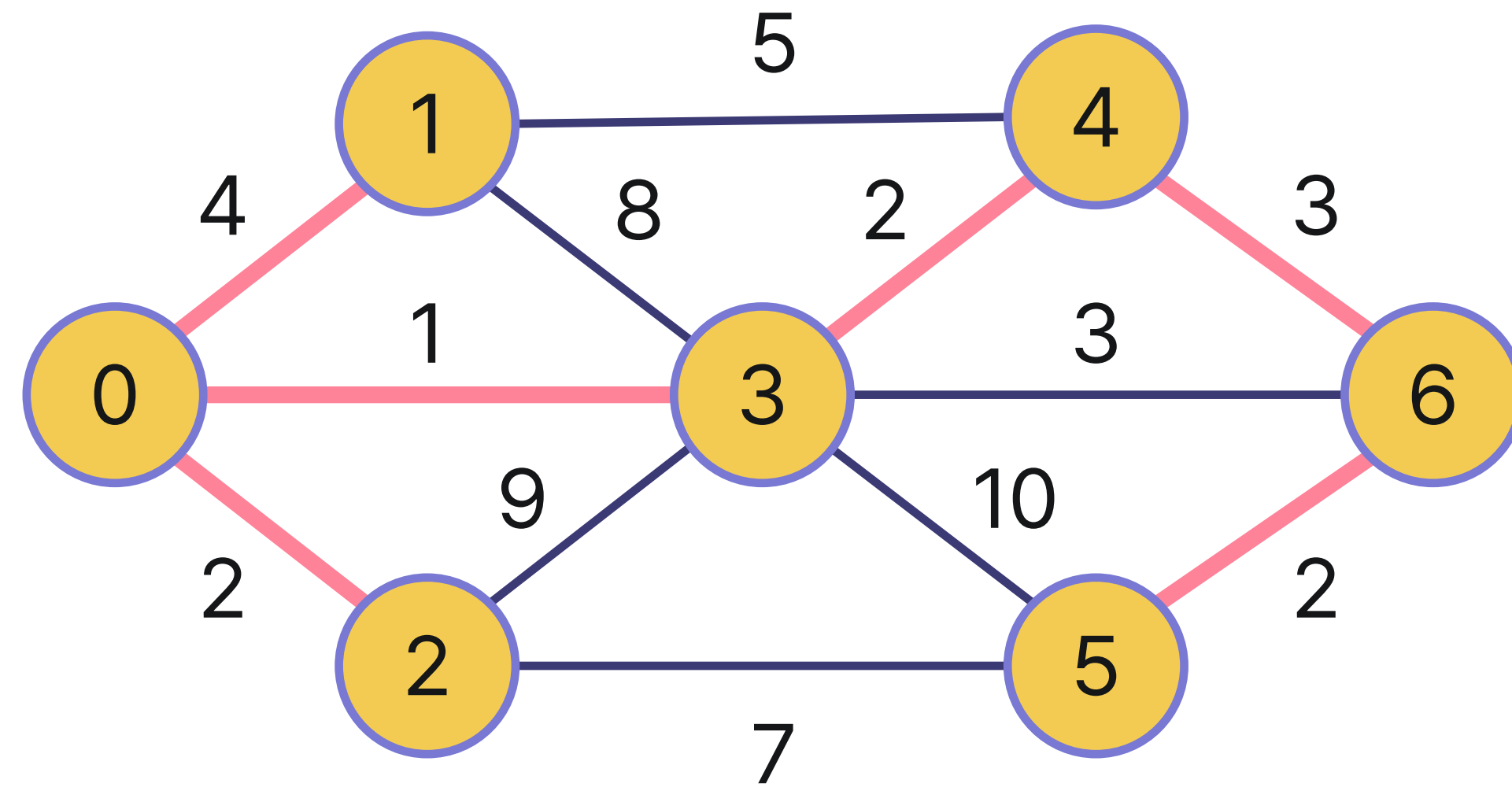
작은 트리에서 연결되어있는 간선들을 하나씩 확장하면서 트리를 만듭니다.





프림 (Prim)

작은 트리에서 연결되어있는 간선들을 하나씩 확장하면서 트리를 만듭니다.





N개의 정점과 M개의 간선으로 이루어진 무방향 그래프가 주어질 때, 주어진 그래프의 모든 정점들을 가중치의 합이 최소가 되도록 MST를 만드려고 한다.

위 조건대로 모든 정점들을 연결하였을 때의 가중치를 출력하는 프로그램을 작성해 보자.



1. 시작 정점만을 MST 집합에 포함

```
n, m = map(int, input().split())
v = [[] for _ in range(n + 1)]
visited = [False] * (n + 1)

for _ in range(m):
    a, b, c = map(int, input().split())
    v[a].append((b, c))
    v[b].append((a, c))

pq = [(0, 1)]
```




2. 이후, MST 집합에 인접한 정점들 중에서 가중치가 가장 작은 간선으로 연결된 정점을 선택하여 트리를 확장 → 우선순위큐 사용!

3. 2번을 모든 정점이 연결될 때까지 반복

```
sum_cost = 0
while pq:
    cost, now = heapq.heappop(pq)

    if visited[now]:
        continue

    sum_cost += cost
    visited[now] = True

    for next_node, next_cost in v[now]:
        if not visited[next_node]:
            heapq.heappush(pq, (next_cost, next_node))

print(sum_cost)
```



그래프 알고리즘의 활용 정리

- 다양한 문제를 그래프로 표현한 후 대표적인 그래프 문제 유형에 대응하여 풀 수 있습니다.
- 대표적인 그래프 알고리즘에는 **최단거리 알고리즘**과 **최소비용트리**가 있습니다.
- **최단거리 알고리즘** 중 하나의 정점에서 다른 모든 정점으로의 최단거리를 구하는데 **다익스트라**, **벨만포드** 등의 알고리즘이 있습니다.
 - 벨만포드는 음의 가중치를 허용하는 대신 느립니다.
- **최소비용트리**는 모든 정점을 최소비용으로 잇는 트리를 말하는데 **프림**, **크루스칼** 등의 알고리즘으로 구현할 수 있습니다.